

1.3.2 Helpful Functions

Show Lecture Video

Python comes with a lot of helpful functions, and we have added a few more for you. We will not cover all of Python's functions, nowhere near. And we do not want you to search for just-the-right-Python function to simplify exercises. Instead, we will cover a subset of Python functions that give you enough power to aid in your programming, while still leaving important logic for you to write on your own. That is critical. The main outcome of this course is not for you to know all the vast details of Python! Rather, it is for you to be very comfortable writing your own solutions to new problems you encounter in the future. So use Python's functions, yes, but sparingly. Gain comfort and confidence in writing your own functions. With that, let's look at the Python functions we encourage you to use throughout the course.

Builtin Functions

Let's start with some builtin Python functions.

type and isinstance

We've already seen that we can use `type(value)` to find the type of a value. We can also use `isinstance(value, t)` where `t` is a type to check if a value has a specific type. For example:

```
1 s = 'abc'
2 print(type(s) == str)
3 print(isinstance(s, str))
4 print(isinstance(s, int))
```

Run

Type Conversion Functions

Run this code (of course, first predicting its output):

```
1 s = '12'
2 print(type(s))
3 print(s * 5)
```

Run

When we run the code, we see that `'12' * 5` is `'1212121212'`. Surprised? Since `'12'` is a string, `*` does not multiply like an int, but instead makes a new string with that many copies of the original string.

Let's change the code a bit:

```
1 n = int('12')
2 print(type(n))
3 print(n * 5)
```

Run

Instead of using `'12'`, which is a string, we used `int('12')`, which is an integer. The builtin function `int()` converts its argument into an integer.

Edit the code above and experiment with other arguments to `int` besides `'12'`, then answer these questions:

Question: What is `int(12.8)`?

Show answer

Question: What is `int('two')`?

Show answer

Each type has its own type conversion function. For example, `float('2.5')` is `2.5`, and `str(2.5)` is `'2.5'`. Perhaps the most surprising type conversion function is `bool()`. This function returns `False` for `0`, `0.0`, and `''` (the empty string), and `True` for non-zero numbers and non-empty strings.

Basic Math Functions

Python includes a few builtin basic math functions:

```
1 print(abs(-5))
2 print(max(2, 3))
3 print(min(2, 3))
```

Run

Again, experiment with the code, then answer these questions:

Question: What does `abs()` do?

Show answer

Question: Is `abs(-12)` the same as `abs(-12.0)`?

Show answer

Question: What is `min(2, 3.5)` and `max(2, 3.5)`?

Show answer

Also note that both `min()` and `max()` can take more than two arguments. For example, `min(5, 1, 7)` returns 1.

Some Builtins To Avoid

Python includes some functions we would want you to avoid. First, there's `pow()`:

```
1 print(pow(2, 3))
```

Run

See how `pow(2, 3)` is the same as `2 ** 3`. This duplication of functionality is unnecessary. Stick with `**` to exponentiate.

Even worse is the builtin `round()`:

```
1 print(round(1.5))
2 print(round(2.5))
```

Run

Run the code above and carefully look at the output. Surprised? Python's builtin `round()` function does not round to the nearest integer in all cases. When `round()` is called on a value that is halfway between two integers, such as 1.5 or 2.5, the value is rounded to the nearest even integer. There are good reasons for that, but still, it is unexpected. And unexpected results should be avoided. So you should not use the builtin `round()` function. In fact, our linter will not accept code that uses it! Of course, sometimes you do have to round, so we will provide you with a useful alternative (in the CPCS Utils Functions section, just below).

Module Functions

Python includes some helpful functions in the `math` module:

```
1 import math
2 print(math.floor(3.9))
3 print(math.ceil(3.2))
```

Run

If n is a float, between two integers, then `math.floor(n)` and `math.ceil(n)` (for *ceiling*) return the integers just below and just above n .

Question: If n is an integer, what is `math.floor(n)`?

Show answer

There are other modules with useful functions, such as `random`. We will discuss those later.

Checkpoint 1

Which of the following expressions evaluates to the largest number?

- ☐ `float(1 * '4')`
- ☐ `int('1' * 3)`
- ☐ `int('4') * '3'`
- ☐ `float('1' + '3')`

Submit

CPCS Utils Functions

At times, we will provide you with some additional helpful functions. We will place these in the `cmu_cpcs_utils` module ("utils" is short for "utilities").

almostEqual()

Recall that you should never use `==` with floats. If either x or y is a float, then instead of using `x == y`, you should use `almostEqual(x, y)`, like so:

```
1 from cmu_cpcs_utils import almostEqual
2 print(0.1 + 0.2 == 0.3)           # False
3 print(almostEqual(0.1 + 0.2, 0.3)) # True (whew!)
```

Run

You may come across `math.isclose()`, which works similarly to `almostEqual`. We recommend you do not use `math.isclose()` as this example shows:

```
1 import math
2 print(math.isclose(0.1 + 0.2, 0.3)) # True, so
3 print(math.isclose(0.1 + 0.2 - 0.3, 0)) # False (
4
5 # Verify this works with almostEqual:
6 from cmu_cpcs_utils import almostEqual
7 print(almostEqual(0.1 + 0.2 - 0.3, 0)) # True (w
```

Run

The problem is that `math.isclose()` does not work as you would expect for values near 0. Just use `almostEqual` and you will be fine.

rounded()

Also recall that you should not use the builtin `round()` function. So how do you round a number? For that, you should use the `rounded()` function, like so:

```
1 from cmu_cpcs_utils import rounded
2 print(round(2.5)) # 2 (yuck!)
3 print(rounded(2.5)) # 3 (yes!)
```

Run

For code that you run outside of CMU CS Academy, we will provide instructions in our docs on how to include the helpful function in `cmu_cpcs_utils`.

Checkpoint 2

For which of the following values does `math.ceil(value) == rounded(value)` ?

- ☐ **2.25 and 2.5**
- ☐ **2.25**
- ☐ **2.5 and 2.75**
- ☐ **2.5**
- ☐ **2.75**

Submit

Checkpoint 3

True or False: `math.isclose` and `almostEqual` work identically, just like `pow` and `**` do.

- ☐ **True**
- ☐ **False**

Submit

Continue